

# 20 exercices d'algorithmique en JavaScript

Parcours progressif pour débutante — exécution et interaction directement dans le terminal, sans HTML ni CSS.

## Objectif

Apprendre progressivement les bases de l'algorithmique avec JavaScript : variables, entrées utilisateur, conditions, boucles, chaînes de caractères, tableaux et résolution de petits problèmes.

## Préparation de l'environnement

Créer un dossier de travail, initialiser Node.js et installer nodemon :

```
mkdir exercices-algorithme
cd exercices-algorithme
npm init -y
npm install --save-dev nodemon
```

Chaque exercice doit être placé dans un fichier séparé : exercice01.js, exercice02.js, ... exercice20.js.

Pour lancer et tester un exercice :

```
npx nodemon exercice01.js
```

Les saisies et les résultats doivent être visibles dans le terminal. Pour les exercices interactifs, utiliser par exemple le module readline de Node.js.

## Méthode de travail

Pour chaque exercice : 1) lire la consigne ; 2) identifier les entrées ; 3) identifier la sortie attendue ; 4) réfléchir à l'algorithme ; 5) écrire le JavaScript ; 6) tester dans le terminal ; 7) corriger les erreurs ; 8) expliquer pourquoi la solution fonctionne.

## 1. Présentation de l'utilisateur

**Consigne :** Écris un programme qui demande à l'utilisateur son prénom et son âge. Le programme affiche ensuite une phrase de présentation.

**Exemple d'exécution :**

```
Quel est ton prénom ? Alice  
Quel âge as-tu ? 19
```

```
Bonjour Alice, tu as 19 ans.
```

**Contraintes / objectif :** Utiliser des variables. Les informations doivent être saisies depuis le terminal. Ne pas écrire directement le prénom ou l'âge dans le code.

## 2. Addition de deux nombres

**Consigne :** Écris un programme qui demande deux nombres à l'utilisateur puis affiche leur somme.

**Exemple d'exécution :**

```
Premier nombre : 12  
Deuxième nombre : 8
```

```
Résultat : 20
```

**Contraintes / objectif :** Les valeurs saisies depuis le terminal sont initialement des chaînes de caractères : pense à les convertir en nombres.

## 3. Mini calculatrice

**Consigne :** Demande deux nombres puis affiche leur addition, soustraction, multiplication et division.

**Exemple d'exécution :**

```
Premier nombre : 10  
Deuxième nombre : 5
```

```
Addition : 15  
Soustraction : 5  
Multiplication : 50  
Division : 2
```

**Contraintes / objectif :** Gérer le cas où le deuxième nombre vaut 0 afin d'éviter une division incorrecte.

## 4. Convertisseur de température

**Consigne :** Demande une température en degrés Celsius et convertis-la en Fahrenheit. Formule :  $F = (C \times 9 / 5) + 32$ .

**Exemple d'exécution :**

```
Température en Celsius : 25
```

```
25°C = 77°F
```

**Contraintes / objectif :** Le programme doit fonctionner avec différentes températures saisies dans le terminal.

## 5. Pair ou impair

**Consigne :** Demande un nombre entier à l'utilisateur et indique s'il est pair ou impair.

**Exemple d'exécution :**

Entrez un nombre : 17

17 est impair.

**Contraintes / objectif :** Utiliser l'opérateur modulo %.

## 6. Positif, négatif ou zéro

**Consigne :** Demande un nombre et détermine s'il est positif, négatif ou égal à zéro.

**Exemple d'exécution :**

Entrez un nombre : -12

Le nombre est négatif.

**Contraintes / objectif :** Prévoir les trois cas possibles avec des conditions.

## 7. Trouver le plus grand nombre

**Consigne :** Demande deux nombres et affiche le plus grand. Si les deux nombres sont identiques, indique qu'ils sont égaux.

**Exemple d'exécution :**

Premier nombre : 14

Deuxième nombre : 27

Le plus grand nombre est 27.

**Contraintes / objectif :** Utiliser des conditions pour effectuer la comparaison.

## 8. Trouver le plus grand de trois nombres

**Consigne :** Demande trois nombres et détermine lequel est le plus grand.

**Exemple d'exécution :**

Premier nombre : 18

Deuxième nombre : 42

Troisième nombre : 31

Le plus grand nombre est 42.

**Contraintes / objectif :** Interdiction d'utiliser Math.max(). Le résultat doit être trouvé avec des conditions.

## 9. Compter jusqu'à N

**Consigne :** Demande un entier N puis affiche tous les nombres de 1 jusqu'à N.

**Exemple d'exécution :**

Entrez N : 7

1  
2  
3  
4  
5  
6  
7

**Contraintes / objectif :** Utiliser une boucle for ou while.

## 10. Compte à rebours

**Consigne :** Demande un nombre de départ et effectue un compte à rebours jusqu'à 0.

**Exemple d'exécution :**

Nombre de départ : 5

5  
4  
3  
2  
1  
0  
Terminé !

**Contraintes / objectif :** Utiliser une boucle décroissante.

## 11. Table de multiplication

**Consigne :** Demande un nombre et affiche sa table de multiplication de 1 à 10.

**Exemple d'exécution :**

Quelle table veux-tu afficher ? 7

7 x 1 = 7  
7 x 2 = 14  
7 x 3 = 21  
...  
7 x 10 = 70

**Contraintes / objectif :** Le programme doit fonctionner avec n'importe quel nombre saisi.

## 12. Calculer la somme de 1 à N

**Consigne :** Demande un entier positif N puis calcule la somme  $1 + 2 + 3 + \dots + N$ .

**Exemple d'exécution :**

Entrez N : 5

$1 + 2 + 3 + 4 + 5 = 15$

**Contraintes / objectif :** Utiliser une boucle et une variable servant d'accumulateur.

## 13. Compter les chiffres d'un nombre

**Consigne :** Demande un entier positif et détermine combien de chiffres il contient.

**Exemple d'exécution :**

Entrez un nombre : 58329

Ce nombre contient 5 chiffres.

**Contraintes / objectif :** Tester le programme avec des nombres de différentes tailles.

## 14. Inverser un nombre

**Consigne :** Demande un nombre et affiche ses chiffres dans l'ordre inverse.

**Exemple d'exécution :**

Entrez un nombre : 12345

Nombre inversé : 54321

**Contraintes / objectif :** Bonus : essayer une solution avec %, des divisions et une boucle plutôt qu'une inversion directe de chaîne.

## 15. Compter les voyelles

**Consigne :** Demande une phrase et compte le nombre de voyelles qu'elle contient. Considère a, e, i, o, u et y comme voyelles.

**Exemple d'exécution :**

Entrez une phrase : Bonjour tout le monde

Nombre de voyelles : 8

**Contraintes / objectif :** Le programme doit gérer les majuscules et les minuscules.

## 16. Détecter un palindrome

**Consigne :** Demande un mot et détermine s'il se lit de la même manière dans les deux sens.

**Exemple d'exécution :**

```
Entrez un mot : radar
```

```
"radar" est un palindrome.
```

**Contraintes / objectif :** Pour commencer, essayer une solution avec une boucle et des indices sans utiliser `reverse()`.

## 17. Calculer la moyenne de plusieurs notes

**Consigne :** Demande combien de notes l'utilisateur souhaite saisir. Demande ensuite chaque note une par une, stocke-les dans un tableau et calcule la somme et la moyenne.

**Exemple d'exécution :**

```
Combien de notes ? 4
Note 1 : 12
Note 2 : 15
Note 3 : 9
Note 4 : 16
```

```
Somme : 52
Moyenne : 13
```

**Contraintes / objectif :** Utiliser une boucle et un tableau. Bonus : afficher combien de notes sont supérieures ou égales à 10.

## 18. Trouver le minimum et le maximum

**Consigne :** À partir du tableau `const nombres = [12, 5, 8, 27, 3, 19, 42, 7]`, trouve le plus petit et le plus grand nombre en parcourant le tableau.

**Exemple d'exécution :**

```
Minimum : 3
Maximum : 42
```

**Contraintes / objectif :** Interdiction d'utiliser `Math.min()` et `Math.max()`.

## 19. Rechercher un nombre dans un tableau

**Consigne :** Utilise le tableau `[4, 8, 2, 8, 5, 8, 10, 3]`. Demande à l'utilisateur quel nombre il souhaite rechercher puis compte le nombre d'occurrences.

**Exemple d'exécution :**

```
Quel nombre recherchez-vous ? 8
```

```
Le nombre 8 apparaît 3 fois.
```

**Contraintes / objectif :** Si le nombre n'existe pas, l'indiquer. Bonus : afficher également les positions où il a été trouvé.

## 20. Jeu : deviner le nombre

**Consigne :** Crée un jeu dans lequel le programme choisit aléatoirement un nombre entre 1 et 100. L'utilisateur propose des nombres jusqu'à trouver la bonne réponse. Après chaque tentative, indique « Trop petit ! » ou « Trop grand ! ».

### Exemple d'exécution :

```
=== DEVINE LE NOMBRE ===  
J'ai choisi un nombre entre 1 et 100.  
  
Ta réponse : 50  
Trop petit !  
  
Ta réponse : 75  
Trop grand !  
  
Ta réponse : 68  
Bravo ! Tu as trouvé !  
Nombre de tentatives : 3
```

**Contraintes / objectif :** Le programme continue tant que le nombre n'est pas trouvé. Bonus : valider les saisies, limiter à 10 tentatives et proposer de rejouer.

## Règles pour valider un exercice

Un exercice est considéré comme terminé lorsque le programme fonctionne avec plusieurs valeurs de test, que les erreurs simples de saisie ont été vérifiées lorsque c'est pertinent, et que l'élève est capable d'expliquer la logique utilisée.

Le but est de construire le raisonnement algorithmique. Avant de chercher une solution externe, l'élève doit d'abord proposer son propre algorithme, même imparfait.